디지털논리회로설계

VHDL Assignment-2
회로 구현 및 검증 보고서

학번 2022042251 | Donghan Kim
제출일 2026년 10월 29일

제출 범위

강의자료 VHDL_1.pdf의 22페이지「1.4 Problems」에 제시된 5개 문제를 VHDL로 구현하고, Logisim-evolution에서 회로를 구성하였다. 별도의 Questa 테스트벤치로 소스의 조합논리 동작을 검증하였다.

문제	구현 내용	소스 / Logisim 파일
1	4-input AND / OR	logic_test.vhd / logi_test.circ
2	8-to-1 Multiplexer	mux8_1.vhd / mux8_1.circ
3	문자 a,b,c,d,f,g 표시	seg7_letters.vhd / segment.circ
4	25인 과반수 투표기	vote25.vhd / vote.circ
5	13비트 개수 계산과 표시	vote13_display.vhd / vote13_display.circ

검증 결과

5개 설계와 테스트벤치의 Questa 컴파일 결과는 오류 0개, 경고 0개이다. 자동 시뮬레이션 19,106개 사례에서 모든 assertion 검사를 통과하였다.

환경: Logisim-evolution 5.0.0 / Questa Altera Starter FPGA Edition 2025.2 / VHDL-2002

1. 4-input AND / OR

입력 a,b,c,d는 각각 1비트이며, out1은 네 입력의 AND, out2는 네 입력의 OR이다. 입력이 모두 1인 경우에만 out1이 1이고, 하나라도 1이면 out2가 1이다.

구현 코드: src/logic_test.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity logic_test is
    port (
        a, b, c, d : in std_logic;
        out1       : out std_logic;
        out2       : out std_logic
    );
end entity logic_test;

architecture bhv of logic_test is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end architecture bhv;
```

검증 방법과 결과

4비트 입력의 전체 16개 조합을 검사하였다. AND와 OR의 기대값을 각각 확인했으며 모두 통과하였다.

입력 abcd	out1 AND	out2 OR
0000	0	0
1000	0	1
1110	0	1
1111	1	1

1. Logisim 시뮬레이션 결과

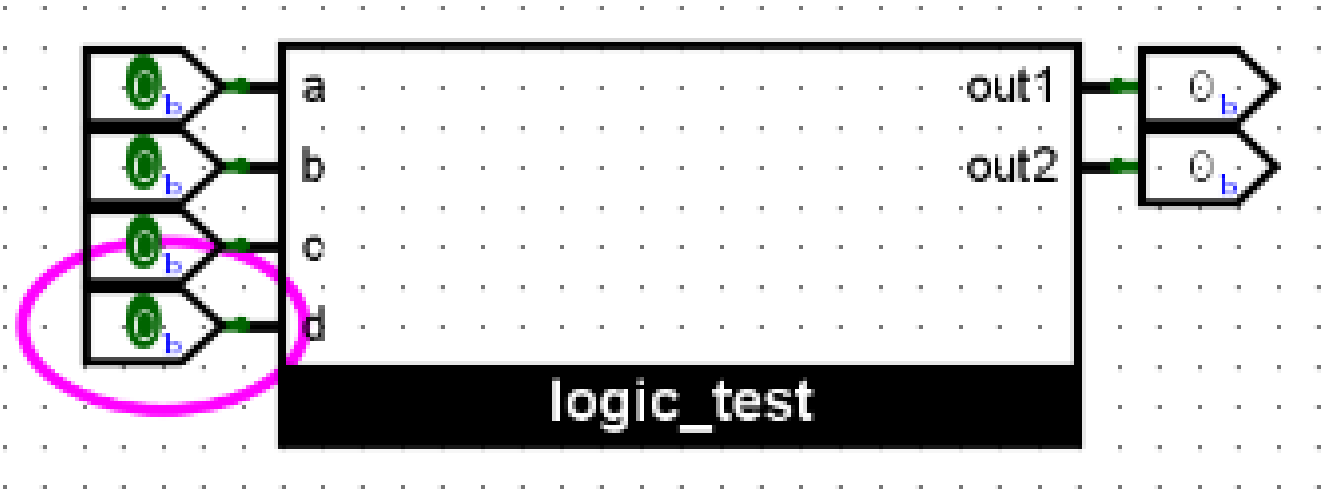


그림 1. 입력 0000에서 AND=0, OR=0. 회로 영역 확대. 원본: screenshots/and or test1 (3).png

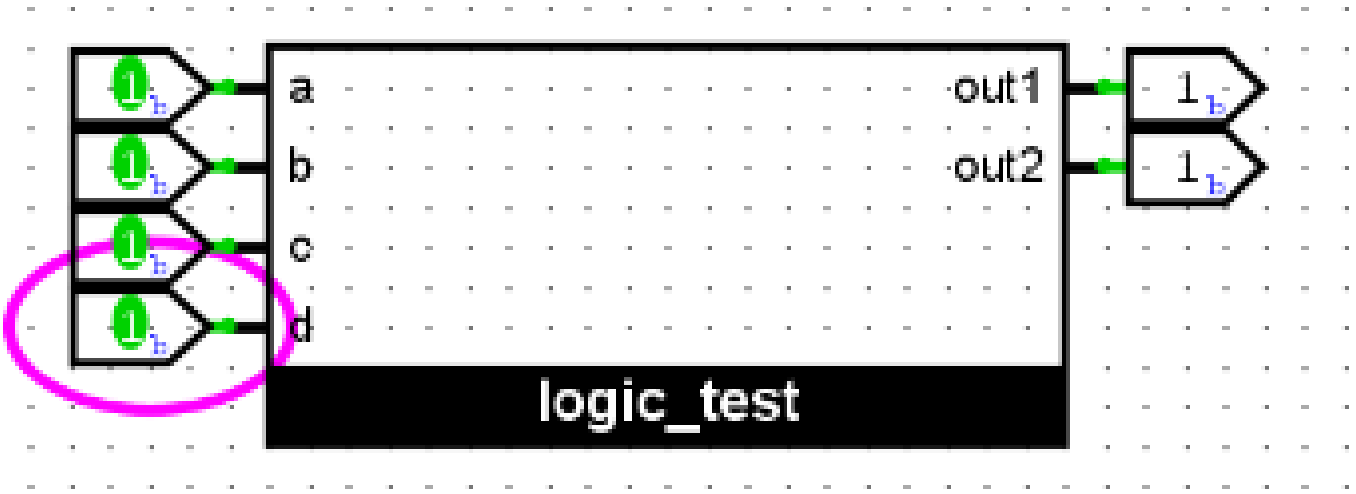


그림 2. 입력 1111에서 AND=1, OR=1. 회로 영역 확대. 원본: screenshots/and or test1 (4).png

2. 8-to-1 Multiplexer

8개의 1비트 입력 a부터 h 중 ch가 지정하는 하나를 data_out으로 전달한다. ch는 3비트 선택 신호이며 000은 a, 001은 b, ..., 111은 h에 대응한다.

구현 코드: src/mux8_1.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity mux8_1 is
    port (
        a, b, c, d, e, f, g, h : in std_logic;
        ch                     : in std_logic_vector(2 downto 0);
        data_out               : out std_logic
    );
end entity mux8_1;

architecture bhv of mux8_1 is
begin
    process(a, b, c, d, e, f, g, h, ch)
    begin
        case ch is
            when "000" => data_out <= a;
            when "001" => data_out <= b;
            when "010" => data_out <= c;
            when "011" => data_out <= d;
            when "100" => data_out <= e;
            when "101" => data_out <= f;
            when "110" => data_out <= g;
            when "111" => data_out <= h;
            when others => data_out <= '0';
        end case;
    end process;
end architecture bhv;
```

검증 방법과 결과

8개 데이터 입력의 256개 패턴과 8개 선택값의 모든 조합, 총 2,048개 사례를 검사하였다. 선택되지 않은 입력값과 관계없이 data_out이 선택된 입력과 같음을 확인하였다.

2. Logisim 시뮬레이션 결과

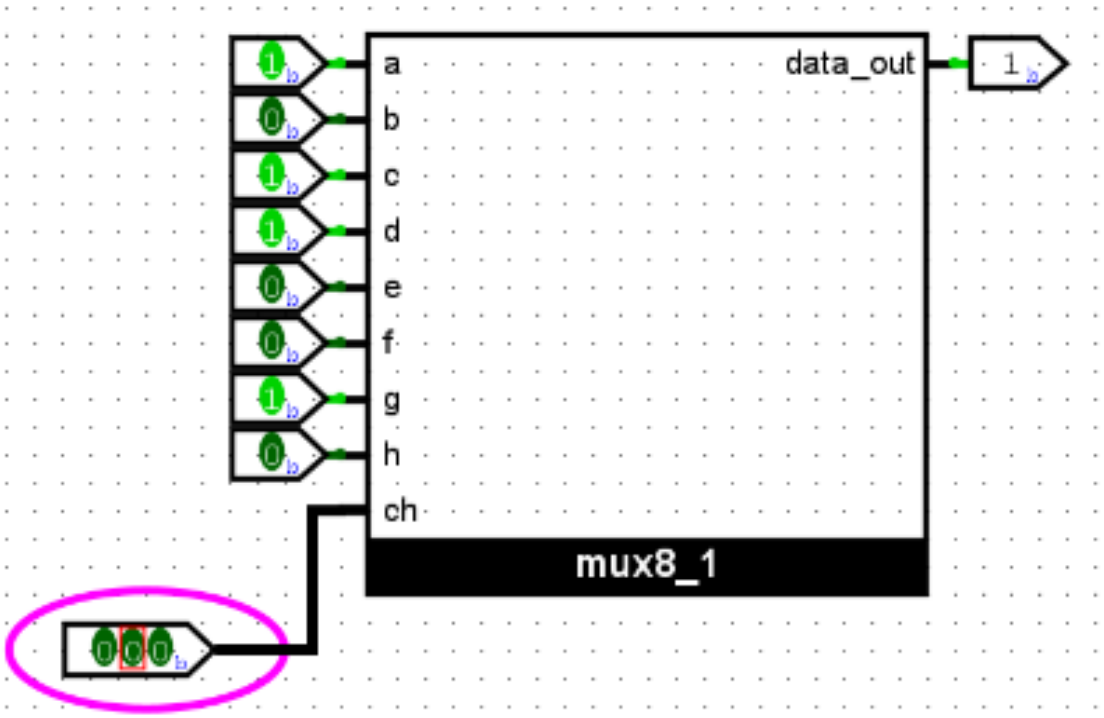


그림 3. ch=000일 때 a=1이 선택되어 data_out=1. 회로 영역 확대. 원본: screenshots/mux (2).png

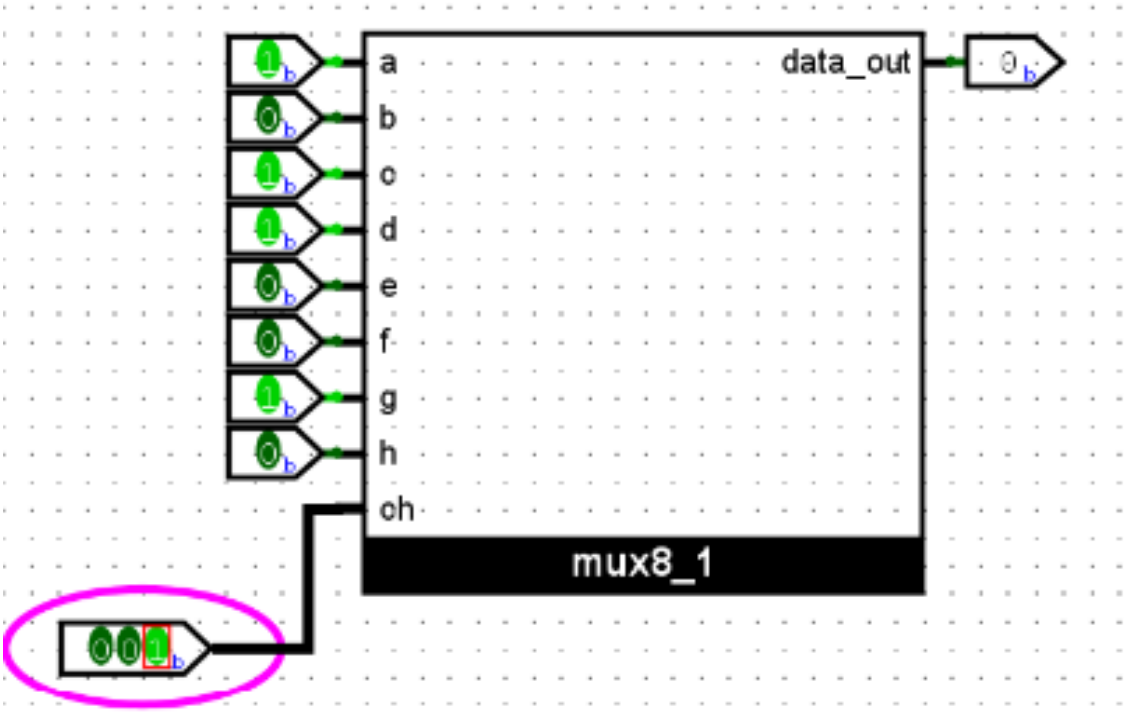


그림 4. ch=001일 때 b=0이 선택되어 data_out=0. 회로 영역 확대. 원본: screenshots/mux.png

3. 7-Segment 문자 표시

3비트 ch로 문자 a,b,c,d,f,g를 선택한다. 출력 segments(7 downto 0)의 비트 순서는 a,b,c,d,e,f,g,dp이다. 각 비트가 1이면 대응 세그먼트를 점등하며 dp는 항상 0이다.

구현 코드: src/seg7_letters.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity seg7_letters is
    port (
        ch       : in  std_logic_vector(2 downto 0);
        segments : out std_logic_vector(7 downto 0)
    );
end entity seg7_letters;

architecture bhv of seg7_letters is
begin
    process(ch)
    begin
        case ch is
            when "000" => segments <= "11111010"; -- a
            when "001" => segments <= "00111110"; -- b
            when "010" => segments <= "00011010"; -- c
            when "011" => segments <= "01111010"; -- d
            when "100" => segments <= "10001110"; -- f
            when "101" => segments <= "11110110"; -- g
            when others => segments <= "00000000";
        end case;
    end process;
end architecture bhv;
```

검증 방법과 결과

ch의 8개 상태를 모두 검사하였다. 000부터 101까지는 순서대로 a,b,c,d,f,g를 표시하며 110과 111에서는 모든 세그먼트를 끈다. 디지털 표시기의 형태에 맞게 소문자 모양을 사용하였다.

ch	표시 문자	출력 Hex
000 / 001	a / b	FA / 3E
010 / 011	c / d	1A / 7A
100 / 101	f / g	8E / F6
110 / 111	꺼짐 / 꺼짐	00 / 00

3. Logisim 시뮬레이션 결과

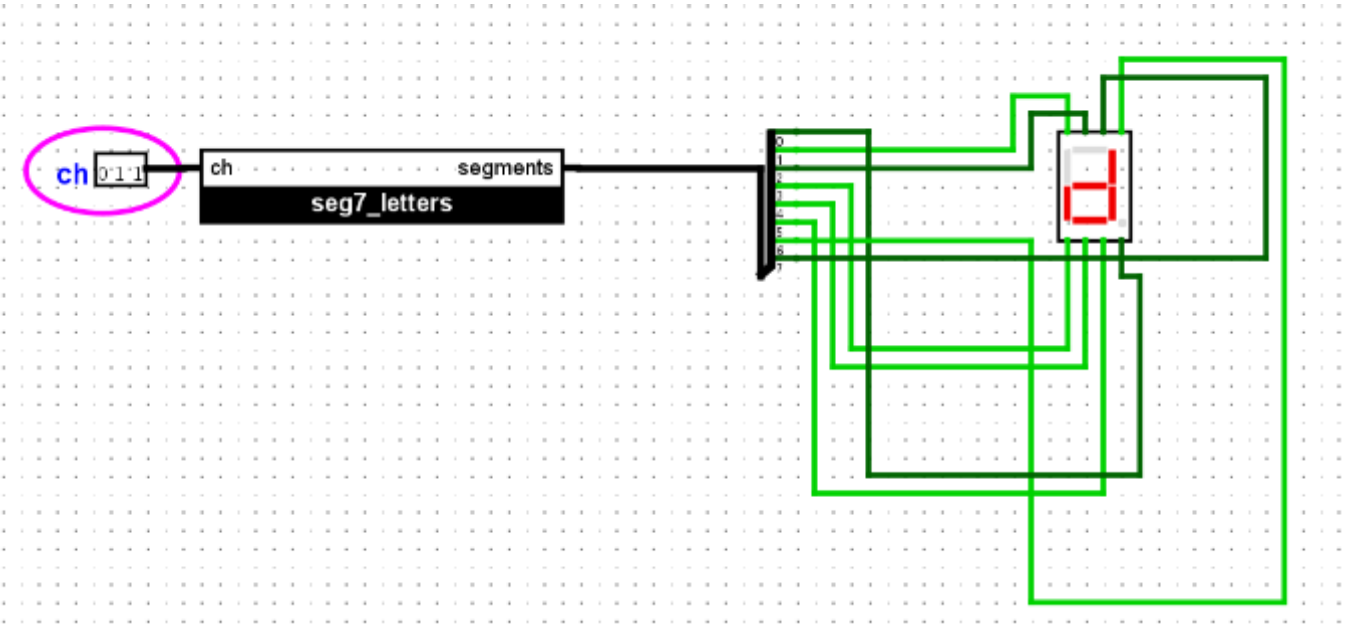


그림 5. ch=011에서 소문자 d 표시. 회로 영역 확대. 원본: screenshots/seg (2).png

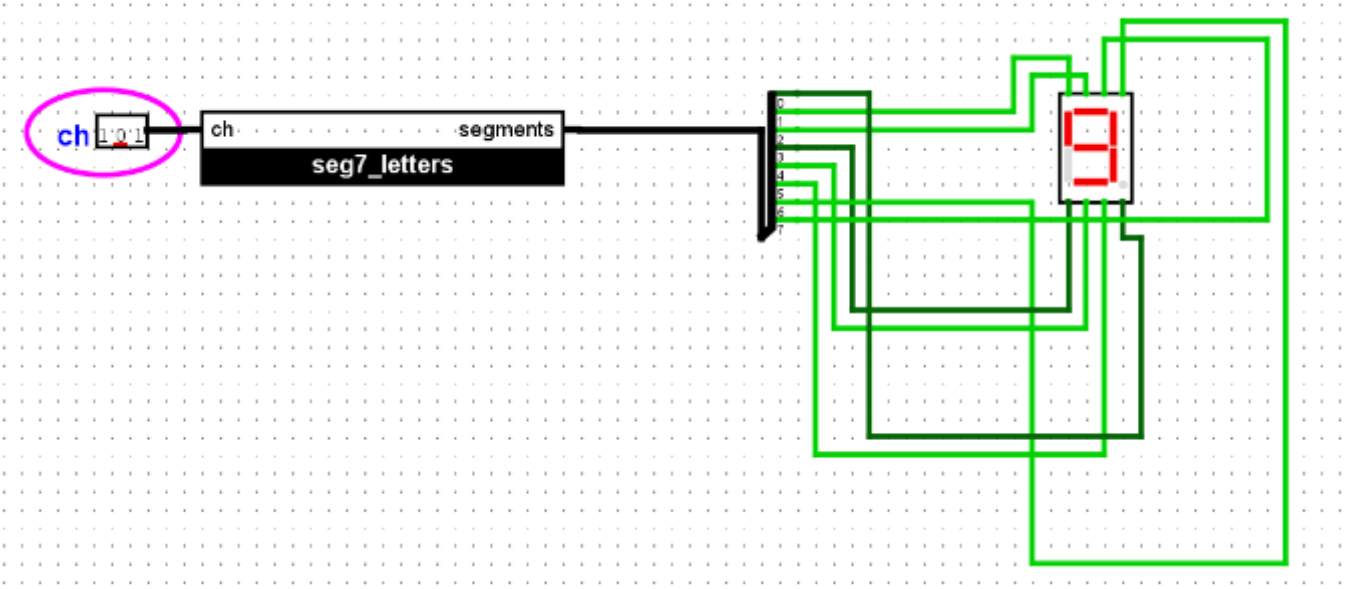


그림 6. ch=101에서 소문자 g 표시. 회로 영역 확대. 원본: screenshots/seg.png

배선 확인: Splitter의 비트 7,6,5,4,3,2,1,0을 각각 a,b,c,d,e,f,g,dp 단자에 연결하였다.

4. 25-person Voting Machine

vt(24 downto 0)의 각 비트는 한 사람의 투표이며 1은 찬성이다. for 반복문으로 1의 개수를 계산하고 13명 이상이면 result=1, 12명 이하면 result=0을 출력한다.

구현 코드: src/vote25.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity vote25 is
    port (
        vt     : in  std_logic_vector(24 downto 0);
        result : out std_logic
    );
end entity vote25;

architecture bhv of vote25 is
begin
    process(vt)
        variable vote_count : integer range 0 to 25;
    begin
        vote_count := 0;

        for i in vt'range loop
            if vt(i) = '1' then
                vote_count := vote_count + 1;
            end if;
        end loop;

        if vote_count >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```

검증 방법과 결과

찬성 수 0부터 25까지 모든 개수에 대해 입력 위치를 25회 회전시킨 총 650개 사례를 검사하였다. 과반수 경계인 12명과 13명뿐 아니라 각 입력 비트가 계수에 반영되는지 확인하였다. 25비트의 전체 33,554,432개 조합을 전수 검사한 것은 아니다.

4. Logisim 시뮬레이션 결과



그림 7. vt=0000FFF, 찬성 12명에서 result=0. 회로 영역 확대. 원본: screenshots/vote.png

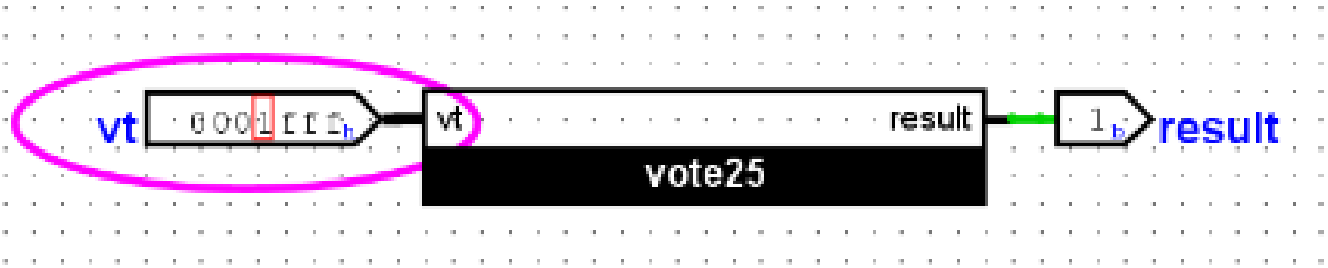


그림 8. vt=0001FFF, 찬성 13명에서 result=1. 회로 영역 확대. 원본: screenshots/vote (2).png

5. 13-bit Voting Count + 7-Segment

vt(12 downto 0)에서 1의 개수를 계산하여 0~9는 숫자, 10~13은 A,b,C,d로 표시한다. en=0이면 display=00000000으로 표시기를 끄고, en=1이면 계수 결과를 표시한다. 13비트 입력이므로 최대 개수는 13이며 E(14), F(15)는 발생하지 않는다.

구현 코드: src/vote13_display.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity vote13_display is
    port (
        vt      : in  std_logic_vector(12 downto 0);
        en      : in  std_logic;
        display : out std_logic_vector(7 downto 0)
    );
end entity vote13_display;

architecture bhv of vote13_display is
begin
    process(vt, en)
        variable vote_count : integer range 0 to 15;
    begin
        vote_count := 0;

        for i in vt'range loop
            if vt(i) = '1' then
                vote_count := vote_count + 1;
            end if;
        end loop;

        if en = '0' then
            display <= "00000000";
        else
            -- Bit order: a b c d e f g dp
            case vote_count is
                when  0 => display <= "11111100"; -- 0
                when  1 => display <= "01100000"; -- 1
                when  2 => display <= "11011010"; -- 2
                when  3 => display <= "11110010"; -- 3
                when  4 => display <= "01100110"; -- 4
                when  5 => display <= "10110110"; -- 5
                when  6 => display <= "10111110"; -- 6
                when  7 => display <= "11100000"; -- 7
                when  8 => display <= "11111110"; -- 8
                when  9 => display <= "11110110"; -- 9
                when 10 => display <= "11101110"; -- A
                when 11 => display <= "00111110"; -- b
                when 12 => display <= "10011100"; -- C
                when 13 => display <= "01111010"; -- d
                when 14 => display <= "10011110"; -- E
                when 15 => display <= "10001110"; -- F
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

5. Logisim 시뮬레이션 결과

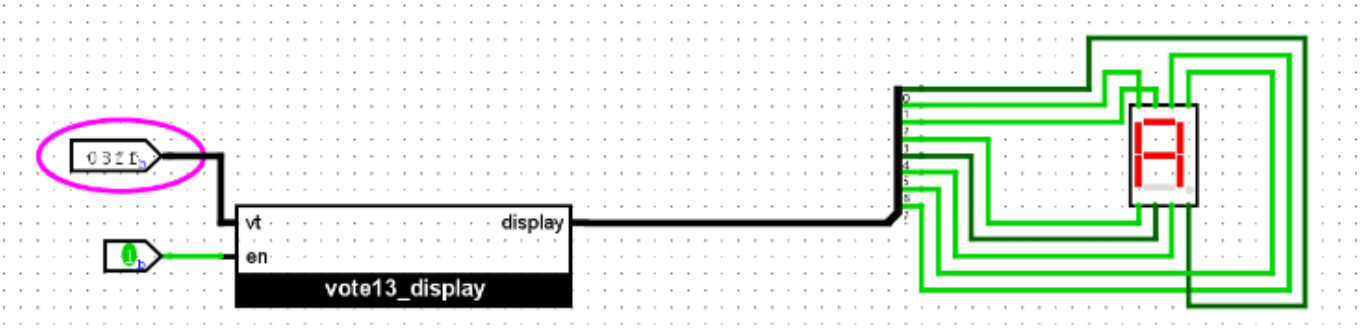


그림 9. vt=03FF, 찬성 10명에서 A 표시(en=1). 회로 영역 확대. 원본: screenshots/vote_display (2).png

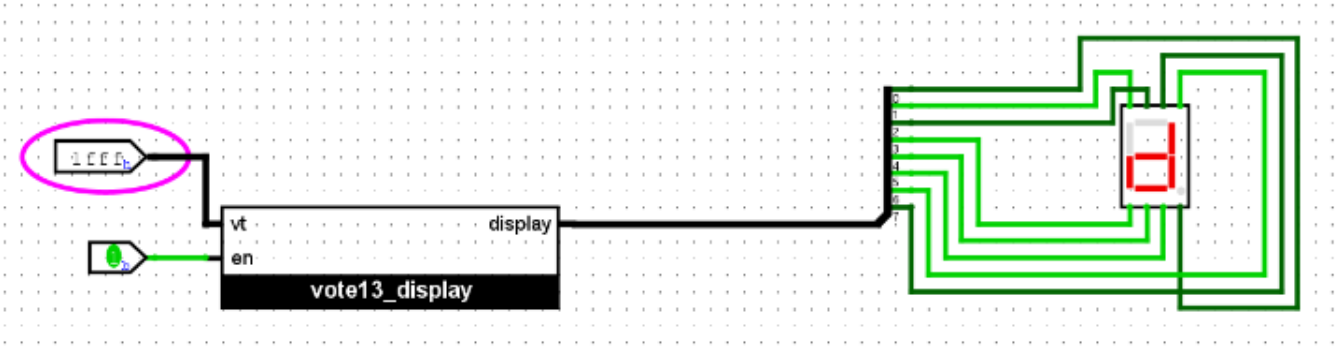


그림 10. vt=1FFF, 찬성 13명에서 d 표시(en=1). 회로 영역 확대. 원본: screenshots/vote_display.png

검증 방법과 결과

13비트 입력의 모든 8,192개 조합을 en=0과 en=1에서 각각 검사하였다. 총 16,384개 사례에서 비활성 출력 00과 활성 출력의 표시 패턴이 기대값과 일치하였다. 스크린샷은 활성 상태의 대표 사례이며 비활성 상태는 자동 테스트 로그로 확인하였다.

6. 검증 기록 및 제출 파일

자동 시뮬레이션 범위

설계	검사 범위	사례 수
logic_test	4비트 입력 전체	16
mux8_1	256개 데이터 패턴 x 8개 선택값	2,048
seg7_letters	6개 문자와 2개 꺼짐 상태	8
vote25	26개 찬성 수 x 25개 위치 회전	650
vote13_display	8,192개 입력 x 2개 enable 상태	16,384
합계	모든 assertion 통과	19,106

실제 실행 로그 발췌

```
** Note: PASS logic_test: 16 input combinations
** Note: PASS mux8_1: 2048 data/select combinations
** Note: PASS seg7_letters: 8 selections, including 2 blank states
** Note: PASS vote25: 650 cases (26 populations x25 rotations)
** Note: PASS vote13_display: 16384 cases (8192 inputs x2 enable states)
** Note: ALL TESTS PASSED: 19106 cases
Errors: 0, Warnings: 0
```

컴파일 로그: validation/compile.log
시뮬레이션 로그: validation/simulation.log
재현용 테스트벤치: validation/tb_assignment2.vhd

파일 구성

src/에는 5개 VHDL 소스가 있으며 circuits/에는 각 문제의 Logisim 프로젝트가 있다. screenshots/에는 실습 중 저장한 원본 캡처가 있고, validation/에는 별도 테스트벤치와 실행 로그가 있다. 이 보고서는 실제 소스와 캡처, 실행 로그를 기준으로 작성하였다.

참고 자료

수업 강의자료 VHDL_1.pdf, 22페이지,「1.4 Problems」.
프로그램: Logisim-evolution 5.0.0, Questa Altera Starter FPGA Edition 2025.2.